RHEL System call mmap

The mmap system call is a powerful tool in Red Hat Enterprise Linux (RHEL) for memory management and file I/O. Its primary purpose is to create a new mapping in the virtual address space of a process.

This mapping can be used for several key tasks:

- File Mapping: Map a file (or part of one) directly into memory, allowing you to access the file's contents as if it were an array in memory.
- Anonymous Memory Allocation: Request a block of memory from the kernel not backed by any file, useful for process-private or shared memory (between parent and child) segments.
- Sharing Memory: The flags parameter controls whether changes to the mapped memory are visible to other processes (MAP_SHARED) or remain private (MAP_PRIVATE).

Understanding the mmap() Syntax

The mmap() function is defined in the <sys/mman.h> header. Its typical declaration looks like this:

void *mmap(void addr[.len], size_t len, int prot, int flags, int fd, off_t offset);

Each of its parameters is crucial:

- void *addr: This is a hint to the kernel about where you'd like to place the mapping. For most applications, providing NULL here and letting the kernel choose is the simplest approach, which provides the best chance of success. Forcing a precise address is possible but rarely necessary.
- size_t len: The length in bytes of the mapping. For file mappings, this must be at least the size of the file segment you wish to map.
- int prot: This sets the memory protection for the mapping. It's a bitwise OR of PROT_NONE, PROT_READ, PROT_WRITE, or PROT_EXEC. A common example is PROT_READ | PROT_WRITE.
- int flags: This defines the mapping's behavior and visibility. The two most important options are MAP_SHARED (changes are visible to other processes that have mapped the

same region) and MAP_PRIVATE (changes are private to this process).

- int fd: This is the file descriptor for the file you want to map into memory. It's obtained by calling open(). This parameter is ignored for anonymous mappings (MAP_ANONYMOUS) when fd should be set to -1.
- off_t offset: The offset from the beginning of the file where the mapping should start. The offset value must be a multiple of the system's page size.

Practical Example: Displaying a File

- Opening the file with open() to get a file descriptor (fd).
- Using fstat() to obtain the file's size (sb.st_size).
- Calling mmap() with NULL for the address, the file's size for the length, PROT_READ for protection, MAP_PRIVATE for flags, the file descriptor, and an offset of 0.
- After a successful call, addr is a pointer that can be used to access the file's data directly in memory, just like any other array.

Implementation

C implementation that demonstrates the mmap() system call on RHEL. The example includes:

1. File mapping – reads a file by mapping it into memory.

2. Anonymous mapping – creates shared memory between a parent and child process.

Both examples include error handling and compilation instructions.

Full Code Example (mmap_demo.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/mman.h>

#include <sys/stat.h>

#include <fcntl.h>

#include <errno.h>
```

```
#include <sys/wait.h>

// Helper function to print errors and exit
void error_exit(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

// Example 1: Map a file into memory and print it (like "cat")
void demo_file_mapping(const char *filename) {
    int fd = open(filename, O_RDONLY);
    if (fd == -1)
        error_exit("open");

    // Get file size
    struct stat st;
    if (fstat(fd, &st) == -1)
        error_exit("fstat");

    // Map the whole file into memory (read-only, private)
    char *data = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (data == MAP_FAILED)
        error_exit("mmap");

    // Write file content to stdout
```

```
    fwrite(data, 1, st.st_size, stdout);
    printf("\n--- End of mapped file ---\n");

    // Clean up
    munmap(data, st.st_size);
    close(fd);
}

// Example 2: Anonymous shared memory between parent and child
void demo_anonymous_shared_memory() {
    // Create an anonymous shared mapping (size: 4KB)
    size_t size = 4096;
    void *shared_mem = mmap(NULL, size, PROT_READ | PROT_WRITE,
                            MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (shared_mem == MAP_FAILED)
        error_exit("mmap anonymous");

    pid_t pid = fork();

    if (pid == -1) {
        error_exit("fork");
    } else if (pid == 0) {
        // Child process: write data into shared memory
        const char *msg = "Hello from child!";
        strcpy(shared_mem, msg);
```

```
        printf("Child: wrote '%s' to shared memory\n", msg);
        munmap(shared_mem, size);  // child can unmap its copy
        exit(EXIT_SUCCESS);
    } else {
        // Parent process: wait for child and read data
        wait(NULL);
        printf("Parent: read from shared memory: '%s'\n", (char *)shared_mem);
        munmap(shared_mem, size);
    }
}

int main(int argc, char *argv[]) {
    // Example 1: file mapping (requires a command-line argument)
    if (argc > 1) {
        printf("=== File mapping demo ===\n");
        demo_file_mapping(argv[1]);
    } else {
        printf("No file provided. Skipping file mapping demo.\n");
    }

    // Example 2: anonymous shared memory between processes
    printf("\n=== Anonymous shared memory demo ===\n");
    demo_anonymous_shared_memory();

    return 0;
```

}